

RC Car Lap Counter

Rishit Madireddy

SYSTEM PLANNING

Problem statement

My cousin has a Scalextric race track but has trouble keeping count of the laps and wants a device to visually represent the Three, Two, One start sequence instead of saying it out loud. This is to essentially remove any biased head-starts at the beginning of the race. The system will be made for 1 track first and then upgraded to 2 if it performs as expected.

Specification

Quantitative:

1. During the start sequence, one red LED should light up for 500ms and then another red LED should light up for another 500ms and finally a green LED should light up for 500ms after which all three turn off.
2. The system should be able to count up to 9 laps (shown on 7-segment display).
3. Once the race finishes, the green LED should flash at a frequency of $1 \text{ Hz} \pm 0.5 \text{ Hz}$.
4. The voltage across each LED should be $2\text{V} \pm 0.5\text{V}$.

Qualitative:

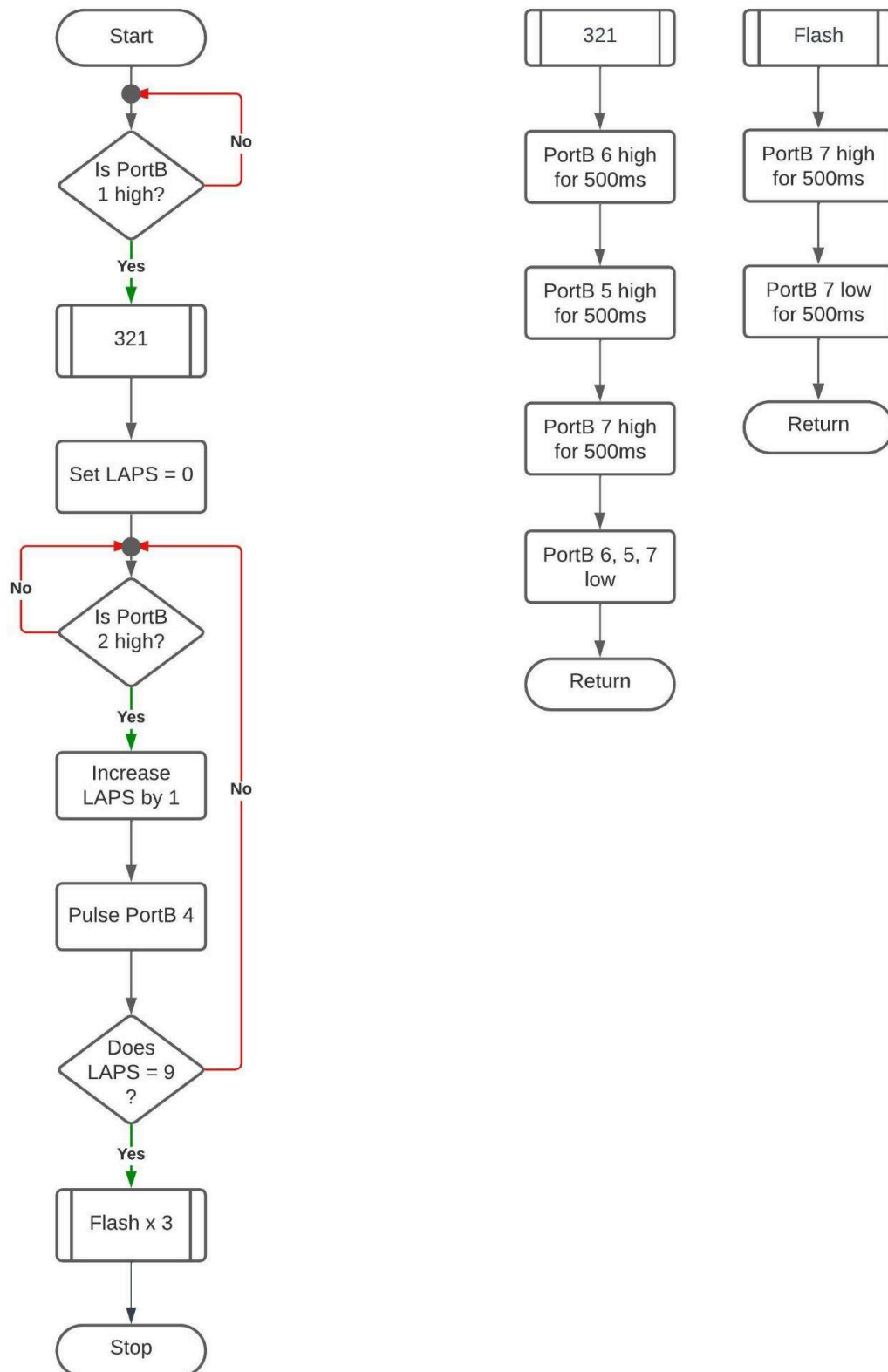
1. The system should include a short starting sequence with the press of a button and count the number of laps completed with a motion sensor (simulated by another switch).
2. The system should be easy to understand given the age of the user with a 7-segment display that isn't dim or hard to read.
3. The system should function as expected with minimal false triggers so that the number of laps is unaffected by any other motion within the motion sensor's range (e.g. shaking table).
4. The system should start by pressing a single button for the starting sequence.

Research

During my research, I found that the average lap time was 6.5 seconds. Since the track has 2 lanes, I plan on using 2 lasers or motion sensors mounted above the start line (one for each lane) which will recognise a complete lap by detecting a break in the laser or motion, respectively. However, for the purpose of my project, I will be testing it with only one laser (simulated by a switch). Given my cousin is 8 years old, I designed my system such that it has only 1 button as an input which minimises accidental triggering or any other problems.

SYSTEM DEVELOPMENT

Flowchart



Predictions on behaviour based on flowchart

Based on the flowchart above, I think that the PIC first checks if Port B1, which is connected to the starting sequence, is on. If it is on, the programme then starts the starting sequence by making a red LED on, a second red LED on and finally a green LED on after which all three LEDs turn off. If Port B1 is not high, the system continues to check. Then, the variable LAPS is initialised and set to 0. After that, when Port B2, which is connected to the motion sensor, is high, LAPS increases by 1 and pulses Port B4, which is connected to the clock input of the 7-segment subsystem. This increments the reading on the 7-segment display by 1. This process of checking if the motion sensor is high continues until LAPS is equal to 9. When LAPS is equal to 9, the flash subroutine is called where the green LED flashes at a frequency of 1 Hz three times. Upon reflection, I also realised that I would need a wait500ms subroutine in between laps so that multiple laps aren't triggered with a single pulse from the motion sensor. This is because if a single PIR is used and placed perpendicular to the track, then based on my research, 500ms gives enough time for the second car to pass the PIR's field of view which means that the score isn't affected by another car's motion.

Analysis of programme

To note, I have provided a full copy of my programme below. Since analysis of my programme was done in MPLAB and then commented out, I have highlighted the respective comments below **like this** so my analysis is clearer and easier to spot.

```
*****
;
;      TEMPLATE PROVIDED BY CENTRE FOR USE WITH HEADER FILE

;      TITLE:   RC Car Lap Counter
;      AUTHOR:  Rishit Madireddy
;      DATE:    13/06/2023

*****
; PROGRAM DESCRIPTION:

; The program is a lap counter for a Scalextric RC car set. First, it will start the race by
; turning on 2 red LEDs
; and a green LED after which it will count up to 9 laps. When the 9 laps have finished, a
; green LED will flash to
; show that the race has ended.

*****
;
;      DEFINITIONS
*****
list      p=16F84A          ; tells the assembler which PIC chip to program for
radix     dec              ; set default number radix to decimal
;radix     hex              ; uncomment this to set radix to hex

#include "p16f84a.inc"

; CONFIG
; __config 0xFFFB
__CONFIG _FOSC_EXTRC & _WDTE_OFF & _PWRTE_OFF & _CP_OFF
```

```

W      EQU h'00'      ; pointer to Working register
F      EQU h'01'      ; pointer to file

;***** REGISTER USAGE *****

;For PIC16F88, user RAM starts at h'20'. The following definitions
;will be found useful in many programs.

; Register page 1
TRISA  EQU h'85'      ; data direction registers
TRISB  EQU h'86'
OSCCON EQU h'8F'      ; internal oscillator speed
ANSEL  EQU h'9B'      ; ADC port enable bits

; Register page 0
STATUS EQU h'03'      ; status
PORTA  EQU h'05'      ; input / output ports
PORTB  EQU h'06'
INTCON EQU h'0B'      ; interrupt control
ADRESH EQU h'1E'      ; ADC result
ADCON0 EQU h'1F'      ; ADC control

;*****
;
;          VARIABLES
;*****
; #### Create any variables here using the EQU statement
; #### Locations H'20' to H'3F' are available for your own use
;
LAPS    EQU h'20'      ; I made a new variable called LAPS which will keep track of how
many laps the car has completed.
WAIT1   EQU h'3C'      ; Counters used in wait delays
WAIT10  EQU h'3D'
WAIT100 EQU h'3E'
WAIT1000 EQU h'3F'

;***** REGISTER BITS *****

C      EQU h'00'      ; carry flag
Z      EQU h'02'      ; zero flag
RP0    EQU h'05'      ; register page bit
INT0IF EQU h'01'      ; interrupt 0 flag
INT0I  EQU h'04'      ; interrupt 0 enable
GIE    EQU h'07'      ; global interrupt enable

;*****
;
;          VECTORS
;*****
;The PIC16F84a reset vectors

      ORG    h'00'      ; reset vector address
      goto   Init      ; goes to first instruction on reset/power-up

;*****
;
;          SUBROUTINES
;*****
; Predefined wait subroutines - wait100ms, wait1000ms

wait100ms
    movlw   d'100'      ; 100 x 1ms = 100ms
    movwf   WAIT100

```

```

wait500ms
    call    wait100ms    ; Since a wait500ms subroutine doesn't exist by default, I
created one by calling five wait100ms subroutines which gives the same result.
    call    wait100ms
    call    wait100ms
    call    wait100ms
    call    wait100ms
    return

wait100ms
    movlw   d'10'        ; 10 x 100ms = 1000ms
    movwf   WAIT1000

;*****
;
;          MAIN PROGRAM
;
;*****

;***** INITIALISATION *****
Init
    CLRF    PORTA        ; Make sure PORTA output latches are low
    CLRF    PORTB        ; Make sure PORTB output latches are low
    BSF     STATUS,RP0    ; Select register page 1

    MOVLW   b'11111111'   ; Since I'm not using any ports in Port A, I set all the port
bits as inputs (0 = output bit, 1 = input bit) which avoids any accidental short circuits.
    MOVWF   TRISA

    MOVLW   b'0000110'    ; There will only be 2 input signals coming into the PIC at
PortB 1 and PortB 2 so I have set the respective ports as inputs.
    MOVWF   TRISB
    BCF     STATUS,RP0    ; Select register page 0

ZeroLaps
    CLRW
    ; The W register has been cleared and the literal value, 0, has
been moved into the variable LAPS.
    MOVLW   0
    MOVWF   LAPS
    goto    Main

;***** MAIN PROGRAM *****
;*****uncomment by removing semicolons from next two lines to use interrupt
routine*****
;bsf     INTCON,INT0IE    ?; set external interrupt enable
;bsf     INTCON,GIE      ?; enable all interrupts

Main
    BTFSC   PORTB, 1      ; When PortB 1 is high, the T10 subroutine is called.
    CALL    T10

    BTFSC   PORTB, 2      ; When PortB 2 is high, a pulse is sent to the 7-segment display
by calling the SSD subroutine.
    CALL    SSD

    MOVLW   b'00001001'   ; This checks if the variable LAPS = 9 which is done by
subtracting the value in LAPS from 9 and checking if the value is 0, in which case the zero
flag is set.
    SUBWF   LAPS, W
    BTFSC   STATUS, Z
    CALL    Flash         ; When the zero flag is set, the Flash subroutine is called 3
times after which the programme goes back to the start.
    CALL    Flash
    CALL    Flash
    GOTO    Main

;*****

```

```

Flash
    BSF    PORTB, 7        ; PortB 7 is connected to a green LED so the bit is high for
500ms and low for another 500ms. When this is called several times, this gives the effect that
the LED flashes.
    CALL   wait500ms
    BCF    PORTB, 7
    CALL   wait500ms
    RETURN

TTO
    BSF    PORTB, 6        ; PortB 6 is connected to a red LED. This causes the bit to
output high so the LED is on for 500ms.
    CALL   wait500ms
    BSF    PORTB, 5        ; After 500ms, PortB 5 outputs high and another red LED
connected to the bit is on for 500ms.
    CALL   wait500ms
    BSF    PORTB, 7        ; A green LED is connected to PortB 7 so after the 2 red LEDs
are one, the bit outputs high and the green LED is on for 500ms.
    CALL   wait500ms
    BCF    PORTB, 6        ; Once the sequence is finished, all 3 LEDs turn off as the
respective port bits are cleared or low.
    BCF    PORTB, 5
    BCF    PORTB, 7
    RETURN

SSD
    BSF    PORTB, 4        ; PortB 4 is connected to the input of a 7-segment display so
this subroutine pulses the input of the 7-segment display by setting the bit high then
immediately low.
    BCF    PORTB, 4
    INCF   LAPS, 1        ; To keep track of the number of laps on the microcontroller
itself, the variable LAPS is increased by 1.
    CALL   wait1000ms     ; Since the system works by detecting motion with the help of a
PIR, there is a possibility for the PIR to record motion twice as the car passes its field of
view.
    RETURN                ; Therefore, I added the wait1000ms subroutine to give
sufficient time for the car to pass the PIR's field of view and so that it isn't triggered
twice.

END

```

Account of assembling the programme

Based on the programme I wrote, I expect the system to first “declare” the variable LAPS and other wait delays. Then, the system will clear the W register and move the literal value 0 into the variable LAPS. After initialisation, the system will check if the input at Port B1 is high. If the input is high, the TTO (Three, Two, One) subroutine is triggered. If the input is low, the TTO subroutine won't be triggered and nothing will happen. The TTO subroutine works by making Port B6 high for 500ms, Port B5 high for another 500ms and finally Port B7 high for 500ms. As 2 red LEDs are connected to Port B5 and Port B6 and a green LED is connected to Port B7, this would give the effect of a start sequence where 2 red lights start one after another and a green light to indicate the race has started. To note, the lights won't turn off after being set high, even after 500ms, as in the next line, I have cleared the necessary bits so they are set low which means that all the lights turn off at the end of the sequence.

Once the race has started, the input at Port B2 will check to see if it goes high (caused by the motion sensor detecting movement). If it is set high, a pulse will be sent to the clock input of the decade counter causing the value on the 7 segment display to increase from 0 to 1. The pulse itself is performed by setting Port B4 high and then immediately low and then incrementing the Variable LAPS by 1. This repeated checking and incrementing is done until the variable LAPS is equal to 9 which is done by subtracting the value in LAPS from 9 and checking if the value is zero, in which case the zero flag is set. I also added a wait 1000ms command as there is a possibility for the PIR to record motion twice as the car passes its field of view. So the wait1000ms subroutine should give sufficient time for the car to pass the PIR's field of view and so that it isn't triggered twice.

If the zero flag is set after checking if LAPS = 9, this indicates the end of the race so the flash subroutine is called three times. This gives the effect that the green LED is flashing as Port B7 is made high for 500ms then low for 500ms, three times.

After the programme finished flashing the LED, I made it go back to the start of the programme by checking if Port B1 was high and so on.

Removal of syntax error

Towards the end of the programme, since the variable LAPS is still equal to 9 from the previous race, I think the 7 segment display will be stuck on 9 and won't increase as the programme doesn't exactly work as a counter where it automatically increases whenever the input is pulsed. To modify this, I would require a goto command to the start of the programme. And since the laps finishes on 9, an initial increment will result in the score being 0 - from which the score can count to 9 as required.

Port bits used

1. Port B1

Port B1 is set as an input bit. This is pin 7 which is connected to a push button switch that initiates the race using the TTO subroutine (Three, Two, One starting sequence).

2. Port B2

Port B2 is set as an input bit. This is pin 8 which is connected to the motion sensor. After the race starts a high input signal is sent when the motion sensor detects motion (i.e. the car passes the motion sensor's field of view).

3. Port B4

Port B4 is set as an output bit. This is pin 10 which is connected to the decade counter and the 7-segment display. When the car completes a lap, pin 10 pulses the clock input on the decade counter which increments the number displayed on the 7-segment display.

4. Port B5

Port B5 is set as an output bit. This is pin 11 and is connected to a red LED which is used in the TTO subroutine. Of the three LEDs used in the TTO subroutine (red, red, green), this port bit triggers the second one.

5. Port B6

Port B6 is set as an output bit. This is pin 12 and is connected to a red LED which is used in the TTO subroutine. Of the three LEDs used in the TTO subroutine (red, red, green), this port bit triggers the first one.

6. Port B7

Port B7 is set as an output bit. This is pin 13 and is connected to a green LED which is used in the TTO and Flash subroutines. Of the three LEDs used in the TTO subroutine (red, red, green), this port bit triggers the third one. And in the Flash subroutine, this port bit is used to turn the LED on for 500ms and then off for 500ms.

Commands used

1. GOTO (Unconditional)
This causes the programme to jump to a different section of the programme without returning and continue to run the programme from the new position.
2. MOVLW (Unconditional)
This moves a literal, constant value into the working register. In my case, I used this to move the value 0 into the working register.
3. MOVWF (Unconditional)
This moves the data from the working register (W) into the file register (F). For example, I used this command to move 0 from the working register into the variable LAPS.
4. CALL (Unconditional)
This calls a specified subroutine and completes the subroutine until the RETURN command is called.
5. RETURN (Unconditional)
This returns to the line after the call command - signifying the end of a subroutine so that the rest of the programme can be executed.
6. CLRF (Unconditional)
This clears the file register (F) so that there is nothing in the file register. I did this in addition to moving 0 with the MOVLW and MOVWF commands.
7. BSF (Unconditional)
This sets the specified bit in the file register as high (1).
8. BCF (Unconditional)
This clears the specified bit in the file register - making it low (0).
9. CLRW (Unconditional)
This clears the working register (W). I used this command so that 0 could be moved into the variable LAPS.
10. BTFSC (Conditional) - Check if Port B1 or Port B2 is high
This command works by checking if the specified bit is set high. I used this to check if Port B1 (connected to the switch controlling the starting sequence) or Port B2 (connected to the motion sensor) is set high.

11. SUBWF (Unconditional)

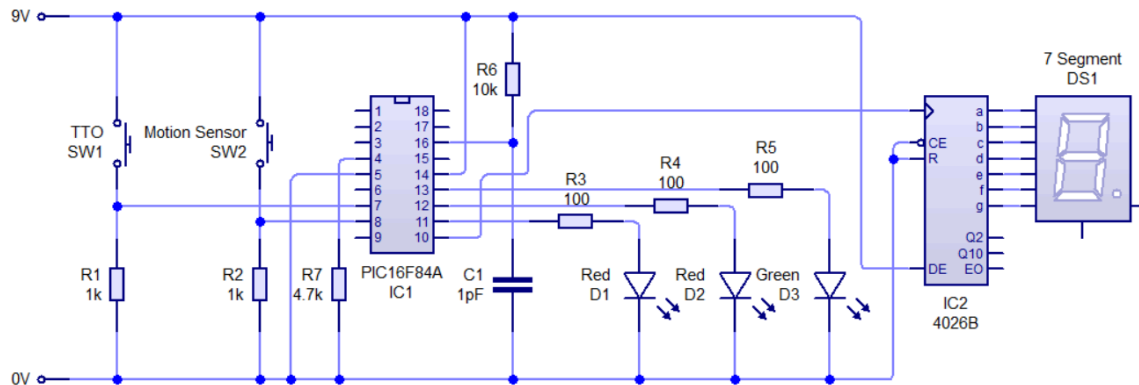
This subtracts the value in the working register (W) from the value in the file register (F). I used this to keep track of the number of laps that the car completes so that if the result is 0, the programme moves on to the next line where the flash subroutine is called three times.

12. INCF (Unconditional)

This increments the value in the file register by 1. I used this to add 1 to the variable LAPS so that every time the motion sensor detects the car, the number of laps increases.

SYSTEM REALISATION

Circuit diagram



Notes:

- SW2 used to simulate motion sensor on Circuit Wizard
- Clock input at Pin 16 on PIC16F84A is 4MHz even though the calculated value using $\frac{1}{2\pi RC}$ is $\approx 16\text{Mhz}$
 - This is because of the following information found on the MPLAB website:

Calculating Frequency

The OSC1 pin is a schmitt trigger input. The charge time to reach the V_{ih} level will control the period of the signal and thus the frequency.

(1)

$$Time = (R_{ext} * C_{ext}) \ln[V_{dd}/(V_{dd} - V_{ih})]$$

(2)

$$F_{osc} = 1/Time$$

(3)

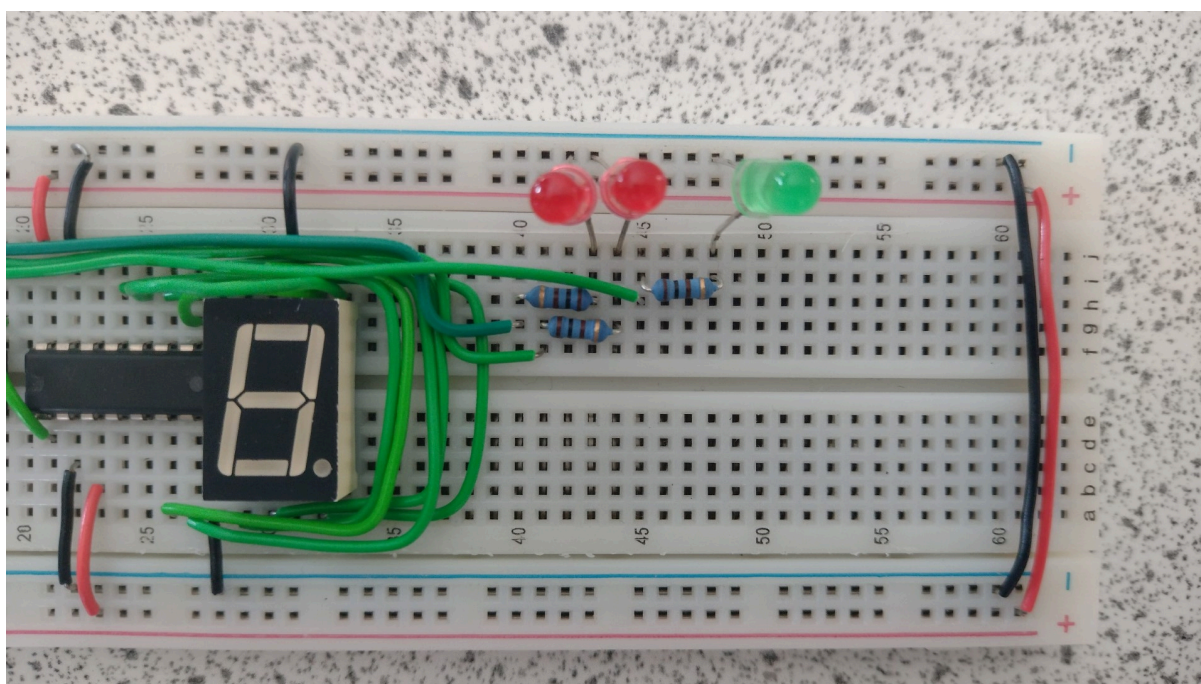
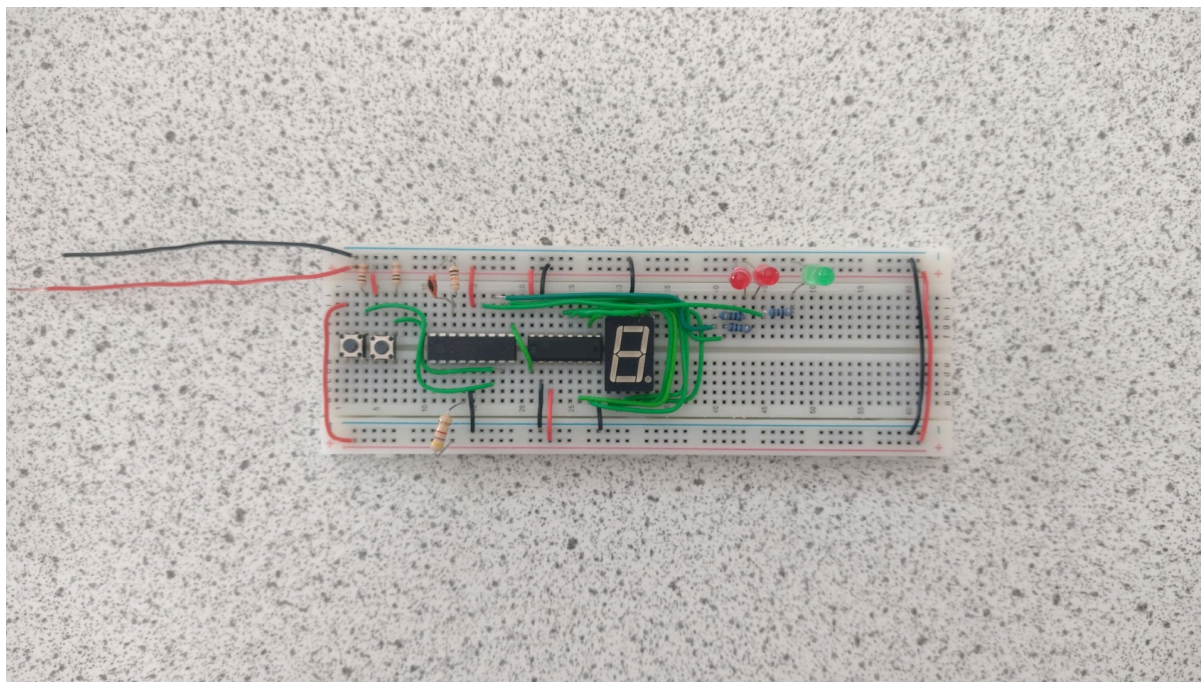
$$InstructionClock = F_{osc}/4$$

- Therefore, $16\text{Mhz} / 4 = 4\text{MHz}$, which is the frequency of the clock input
- The 4.7k resistor, R7 is used to connect the MCLR pin to ground

Component list and schematic labels

Label on schematic	Component name	Part number	Quantity
R1, R2	1K resistor	N/A	2
R3, R4, R5	100Ω resistor	N/A	3
R6	10K resistor	N/A	1
R7	4.7K resistor	N/A	1
SW1	Push button switch	COM-09190	1
SW2	Motion sensor	BS612	1
PIC16F84A	PIC16F84A	PIC16F84A-04/P	1
C1	1pF capacitor	561R10TCCV50	1
IC2	Decade counter	CD4026B	1
DS1	7 Segment display	OSL10561-LR	1
D1, D2	Red LED	HLMP-S100	2
D3	Green LED	HLMP-4740	1

Pictures



Final output test

Specification conditions	Expected result	Actual result	Pass/Fail
Quantitative 1	2 red LEDs and 1 green LED lights up in the order: red, red, green, off	2 red LEDs and 1 green LED lights up in the order: red, red, green, off	Pass
Quantitative 2	Counts up to 9 laps ± 1 lap	Counts to 9 laps	Pass
Quantitative 3	Green LED flashes at 1 Hz ± 0.5 Hz	Green LED flashes at 1.00 Hz	Pass
Quantitative 4	Voltage across green LED = 2V ± 0.5 V	1.805V across green LED	Pass
Qualitative 1	Short start sequence and functions properly	1.5 second starting sequence and system works	Pass
Qualitative 2	User-friendly and easy to understand	Only 1 button so not hard to use and clear, bright 7-segment display	Pass
Qualitative 3	Minimal false triggers	Does not false trigger except for slight movement within very small 15cm range	Pass
Qualitative 4	Starts with 1 button	Works by pressing 1 button for the start sequence	Pass

Analysis of test results

After testing, I found out that the system does the following. First, the starting sequence is initiated when the switch is pressed which triggers a red LED to turn on for 500ms then another red LED for another 500ms and finally a green LED for 500ms. Then, once LAPS has been “declared”, the system counts the number of laps from 0 to 9 by detecting a pulse sent from the motion sensor if there is motion. The counting itself is done by incrementing the value in LAPS by 1 and then waiting 1s (or 1000ms) before another pulse can be detected so that enough time is given for the car to pass clear of the sensor’s field of view, reducing the chance of accidental triggers. To ensure the value in LAPS corresponds with the value displayed on the 7-segment, I made sure that the output of the PIC connecting to the clock input of the 4026 is pulsed whenever the value in LAPS is incremented. As a result, the 7-segment display counts from 0 to 9. When the number of laps is equal to 9, the flash subroutine starts and a green LED flashes to show that the race has ended. The system checks if the value in LAPS is equal to 9 by subtracting the value in LAPS from the value stored in the working register, 9, and checks if the result is 0. If it is zero, the flash subroutine starts, as expected. In the flash subroutine, the green LED is on for 500ms and then off for 500ms, imitating a flash. This repeats 3 times after which the system stops. Therefore, the system works as per the requirements in the specification and this is also shown in the above results table as the programme and system passed with the given conditions.

Evaluation

Improvements

1. As the system only works for one race, I could modify it so that the system works if races are repeated. This can be done by adding a CLRW command for the LAPS variable at the start of my main programme and not just in the initialisation.
2. Another change I can make is adding multiple inputs for multiple motion sensors that can track the laps of cars on different tracks. This would be more efficient than my original plan of creating a whole new system with a different microcontroller as I can use the available bits on Port A and Port B.
3. Another issue I faced was syntax errors. Initially, I mistyped which port bit is connected to the green LED so when I compiled, MPLAB gave me an error as the port bit I told to flash was initialised as an input bit (not an output bit) and hence the programme wouldn’t compile. Hence, I changed the relevant port bit so that the issue was fixed.

Conclusion

Overall, I believe the project is successful. During usage, there may be false triggers as the range of the PIR motion sensors are relatively large compared to the size of the race track. This could mean that an accidental hand movement or shaking the table could cause the laps to increment by 1 even though the car hasn't crossed the line. Also, the system only works for 9 laps so in the future, I could modify the system to include two 7-segment displays so that it can be used for multiple laps. Therefore, as these issues are easily solvable with slight modifications and the testing of the entire system was successful (given that it passed all the points in the specification), I think the system works as expected.

PLAN

System planning

Problem statement

Specification with 2+ realistic measurable parameters with tolerances

Research (e.g. lap time)

System development

Flowchart

Predictions on behaviour

Analyse code - 5+ port bits, 10+ conditional and unconditional commands, removal of syntax error

Add 10K resistor instead of 1K

Add list of components + annotate circuit

System realisation

Circuit diagram

Pictures

Final output test

Evaluation

Compare with specification, 2+ improvements